

Lerneinheit 1 · Entwicklungsumgebung

Version 1.2

Lernziele

Nach diesem Dokument können Sie ...

- eine der Arbeitsumgebungen des Kurses einrichten (Colab, lokal, später der Hochschul-Server) und die Kurs-Notebooks darin öffnen,
- ein Jupyter-Notebook sicher bedienen: Zellen ausführen, den Zustand zurücksetzen, Ihre Kopie sichern,
- mit dem Selbsttest feststellen, was Ihre Umgebung kann.

Dieses Dokument arbeiten Sie einmal zu Semesterbeginn durch und kommen nur zurück, wenn Sie die Umgebung wechseln. Vorweg steht, wie die Kursmaterialien heißen. Teil 1 bringt die Notebooks zum Laufen: drei Wege, eine Variante für IDE-Fans und die Frage, wann Sie eine GPU brauchen (Vorgriff: noch nicht). Teil 2 erklärt die Bedienung von Jupyter-Notebooks — fünf Handgriffe, die den ganzen Kurs tragen. Der Selbsttest am Ende sagt Ihnen, was Ihre Umgebung kann. Kurzfassung für Eilige: Colab-Link aus Moodle öffnen, Kopie in Drive speichern, unten die Prü fzellen ausführen — fertig.

Inhalt

- 1.1 [So heißen die Kursmaterialien](#)
- 1.2 [Weg 1: Google Colab](#)
- 1.3 [Weg 2: Lokal ohne Google-Konto](#)
- 1.4 [Weg 3: Der Hochschul-Server](#)
- 1.5 [GPU und CUDA: Was Sie heute brauchen](#)
- 1.6 [Jupyter-Notebooks bedienen](#)
- 1.7 [Selbsttest: Was kann meine Umgebung?](#)

1.1 So heißen die Kursmaterialien

Jede Lerneinheit besteht aus einem festen Satz von Dateien:

Datei	Was das ist
<code>LExx_workbook.ipynb</code>	das Workbook , das Hauptnotebook der Lerneinheit — zum Mitarbeiten
<code>LExx_skript.pdf</code>	das Skript der Einheit: vollständig zum Nachlesen, inhaltlich gleich zum Workbook
<code>LExx_loesungen.pdf</code>	die Lösungen — erscheinen eine Woche nach dem Termin

Dazu kommen je nach Einheit **Ergänzungsmaterialien** nach dem Muster `LExx_<thema>.*`: eigenständige Notebooks mit eigenem Skript-PDF (etwa `LE01_python_grundlagen_1`), Folien oder Artikel. Dieses Dokument ist selbst eins.

Die Notebooks starten Sie über die Colab-Links in Moodle (Weg 1) oder später auf dem Hochschul-Server (Weg 3). Für die lokale Arbeit (Weg 2) gibt es je Einheit ein ZIP `LExx_material.zip` mit allen Unterlagen, den Daten und den Startskripten.

1.2 Weg 1: Google Colab

Colab ist der Standardweg der ersten Wochen: Die Notebooks laufen im Browser auf Google-Servern, Sie müssen dafür nichts installieren. Colab ist kostenlos, die Gratis-Kontingente (Sitzungsdauer, GPU-Zeit) sind allerdings begrenzt und schwanken je nach Auslastung. Für den Kursumfang reichen sie normalerweise. Sie brauchen ein Google-Konto (die Nutzung ist freiwillig — wer Google nicht nutzen möchte, nimmt einfach Weg 2, siehe unten).

So funktioniert es:

1. In Moodle den Colab-Link der Lerneinheit anklicken — das Notebook öffnet sich direkt in Colab.
2. **Sofort „Kopie in Drive speichern“** (Schaltfläche über dem Notebook). Ohne diesen Klick arbeiten Sie in einer Wegwerf-Ansicht, und Ihre Änderungen sind beim Schließen verloren. Ihre Kopie finden Sie danach jederzeit in Google Drive unter „Colab Notebooks“.
3. Zellen ausführen mit Umschalt+Eingabe. Beim ersten Ausführen fragt Colab einmal, ob Sie dem Notebook vertrauen — ja.

GPU dazuschalten: Menü *Laufzeit* → *Laufzeittyp ändern* → *T4-GPU*. Zwingend ist das in den ersten Lerneinheiten nicht, es lohnt sich aber schon im Appetizer von Lerneinheit 1: Mit GPU-Laufzeit messen Sie dort die PyTorch-GPU-Ebene und das Beispiel zur asynchronen Ausführung live, statt die vorab gemessenen Werte zu sehen; die weiteren GPU-Ebenen folgen in späteren Lerneinheiten.

Wenn Colab die Sitzung trennt (das passiert nach einiger Zeit Inaktivität): Ihre Kopie bleibt erhalten, aber der Rechenzustand ist weg — alle Variablen müssen neu berechnet werden. Führen Sie nach dem erneuten Verbinden die benötigten Zellen einfach noch einmal aus; *Laufzeit* → *Alle ausführen* ist der bequemste Weg.

Drei Handgriffe für Colab

Erstens: Nach dem Öffnen **„Kopie in Drive speichern“** klicken, sonst gehen Ihre Änderungen beim Schließen verloren. Zweitens: Zellen führt **Umschalt+Eingabe** aus. Drittens: Wenn Sie versehentlich in eine Textzelle doppelgeklickt haben und den Bearbeitungsmodus sehen — **Umschalt+Eingabe** stellt die Ansicht wieder her.

1.3 Weg 2: Lokal ohne Google-Konto

Die Notebooks laufen genauso auf Ihrem eigenen Rechner (Windows, macOS, Linux). Das Werkzeug dafür ist **uv**, ein Python-Umgebungsmanager: Er lädt beim ersten Start eine saubere Umgebung mit genau den Kurs-Paketen und lässt Ihr System ansonsten unangetastet.

1. Einmalig uv installieren — den Ein-Zeilen-Befehl für Ihr Betriebssystem finden Sie unter docs.astral.sh/uv (unter „Installation“).
2. Das Kursmaterial aus Moodle laden (`LExx_material.zip`) und entpacken — darin liegen die Notebooks, die Daten und die Startskripte.
3. Das Startskript für Ihr System ausführen. Unter **Windows**: Doppelklick auf `start_lokal.bat`. Unter **macOS**: Doppelklick auf `start_lokal.command` — beim allerersten Mal blockiert macOS die Datei, weil sie aus dem Internet stammt; ein Rechtsklick und „Öffnen“ gibt sie frei, und wenn macOS das nicht anbietet, führt der Weg über *Systemeinstellungen* → *Datenschutz & Sicherheit* → „Dennoch öffnen“. Unter **Linux**: ein Terminal im entpackten Materialordner öffnen und `sh start_lokal.sh` eingeben. Das Skript legt eine Umgebung mit der Kurs-Python-Version (3.12) und den Kurs-Paketen an und öffnet JupyterLab im Browser; der erste Start dauert ein paar Minuten, danach geht es in Sekunden.
4. In JupyterLab links im Dateibrowser das gewünschte Notebook per Doppelklick öffnen — zum Ausprobieren dieses hier, `LE01_entwicklungsumgebung.ipynb`.

Der Kompromiss: kein Google-Konto und offline arbeiten — dafür gibt es lokal meist keine CUDA-GPU (siehe unten), und bei Problemen mit Ihrer Installation können wir nur begrenzt helfen.

Variante für IDE-Fans: VS Code. Auch Visual Studio Code kann die Kurs-Notebooks direkt öffnen — mit den Erweiterungen *Python* und *Jupyter* (beide von Microsoft). Legen Sie im entpackten Materialordner einmal eine Umgebung an: `uv venv --python 3.12`, dann `uv pip install numpy pandas matplotlib ipykernel`. Ordner in VS Code öffnen, Notebook anklicken, oben rechts als Kernel die `.venv` auswählen — fertig. Diese Variante ist aber „unsupported“: Sie funktioniert, aber bei VS-Code-Problemen können wir im Kurs nur begrenzt helfen — dann bitte zurück zum Startskript oder zu Colab.

1.4 Weg 3: Der Hochschul-Server

In wenigen Wochen kommt der Hauptweg des Kurses dazu: ein JupyterHub auf dem GPU-Server der Hochschule. Dann melden Sie sich im Browser mit Ihrem Hochschul-Zugang an (aus dem Hochschulnetz oder per VPN) und haben Notebooks, Daten und echte GPUs an einem Ort — ohne Google-Konto und ohne Installation. Sobald er bereitsteht, erklären wir den Umstieg im Termin; an den Notebooks ändert sich nichts, und dieser Abschnitt wird entsprechend erweitert.

1.5 GPU und CUDA: Was Sie heute brauchen

CUDA ist die Plattform, mit der Programme auf NVIDIA-Grafikkarten rechnen. Das ist der Weg, über den in diesem Kurs später jeder GPU-Code läuft. Eine CUDA-GPU haben Sie also genau dann, wenn in Ihrem Rechner eine NVIDIA-Karte steckt. Alles andere — Macs (auch mit Apple-Chip), AMD- und Intel-Grafik — kann vieles, aber kein CUDA.

Warum ausgerechnet NVIDIA und was CUDA technisch ist, klärt die GPU-Lerneinheit. Für die Einordnung heute reicht: CUDA ist der Industriestandard. PyTorch unterstützt zwar auch andere Backends (etwa Apple-Chips und AMD-Karten), am besten ausgebaut ist aber der CUDA-Pfad, und die großen KI-Rechenzentren setzen fast durchweg auf NVIDIA. Fürs Training von Modellen ist CUDA damit weiterhin die erste Wahl; bei der Inferenz, also dem Ausführen fertiger Modelle, haben Alternativen wie TPUs, AMD und spezialisierte Chips inzwischen echte Anteile.

Und jetzt die wichtigste Nachricht: **Für die ersten Lerneinheiten brauchen Sie noch keine GPU.** Python, NumPy, pandas und die ersten PyTorch-Schritte laufen vollständig auf der CPU. Wenn die GPU-Lerneinheiten kommen, gibt es für jede Situation einen Weg:

Ihre Situation	Ihr GPU-Weg
Rechner mit NVIDIA-Karte	lokal möglich, sofern Grafikkarte, Treiber und eine CUDA-fähige PyTorch-Installation zusammenpassen — der Selbsttest unten sagt es Ihnen
Mac, AMD/Intel-Grafik, Laptop ohne GPU	Colab mit T4-GPU (kostenlos) — oder der Hochschul-Server, sobald er da ist
gar kein eigener Rechner zur Hand	Colab im Browser reicht aus; nur die präzisen GPU-Messungen später im Semester laufen auf dem Hochschul-Server

1.6 Jupyter-Notebooks bedienen

Alle Kursmaterialien sind **Jupyter-Notebooks**: Dokumente aus Zellen, in denen sich Text und ausführbarer Code abwechseln. Textzellen erklären, Codezellen rechnen — Code, Ergebnis und Erklärung stehen beieinander, deshalb sind Notebooks das Standardwerkzeug der Daten- und KI-Welt.

Zellen ausführen. Klicken Sie eine Codezelle an und drücken Sie **Umschalt+Eingabe**: Die Zelle läuft, das Ergebnis erscheint darunter, die Auswahl springt zur nächsten Zelle. **Strg+Eingabe** führt aus, ohne weiterzuspringen. Neben ausgeführten Zellen steht eine Nummer in eckigen Klammern — die Reihenfolge, in der die Zellen gelaufen sind.

```
# Probieren Sie es: Zelle anklicken, Umschalt+Eingabe
antwort = 6 * 7
print("Es läuft! Die Antwort ist", antwort)
```

Es läuft! Die Antwort ist 42

Der Kernel — das Gedächtnis des Notebooks. Hinter jedem Notebook läuft ein Python-Prozess, der **Kernel**. Er merkt sich alles, was Ihre Zellen definieren: `antwort` aus der Zelle oben existiert jetzt auch für jede andere Zelle. Deshalb zählt die **Ausführungs-Reihenfolge**, nicht die Position im Dokument — wer mittendrin einsteigt, dem fehlen oft die Variablen von weiter oben.

Zwei Modi. Ein Klick in eine Zelle bringt Sie in den **Bearbeitungsmodus** (Sie tippen in der Zelle), **Esc** zurück in den **Kommandomodus**, in dem Tasten das Notebook steuern. In **JupyterLab** (lokal und auf dem Hochschul-Server): `a / b` fügt eine Zelle oberhalb/unterhalb ein, zweimal `d` löscht die markierte Zelle. **Colab** hat eigene Kürzel: erst **Strg+M** (Mac: **Cmd+M**), dann `a`, `b` oder `d` — oder Sie nutzen einfach die Schaltflächen, die am Zellenrand erscheinen. Und wenn Sie aus Versehen in eine **Textzelle** doppelgeklickt haben und plötzlich deren Quelltext sehen: **Umschalt+Eingabe** stellt die Ansicht wieder her.

Der Reset-Knopf

Wenn sich ein Notebook seltsam verhält — Variablen fehlen, Ergebnisse passen nicht zur Reihenfolge —, gibt es einen zuverlässigen Neuanfang: Menü *Laufzeit* (Colab) bzw. *Kernel* (JupyterLab) → **Neu starten und alle ausführen**. Das Notebook läuft dann einmal sauber von oben nach unten durch. Eine Ausnahme ist erwartbar: Bei noch ungelösten Aufgaben schlagen deren Prü fzellen fehl — das ist kein Installationsfehler. Führen Sie in dem Fall nach dem Neustart die Zellen bis zu Ihrer aktuellen Aufgabe aus und arbeiten Sie dort weiter.

Speichern. In Colab arbeiten Sie erst dauerhaft, nachdem Sie einmal „**Kopie in Drive speichern**“ geklickt haben — danach speichert Colab automatisch. Die Drive-Kopie sichert dabei nur das Notebook selbst: Dateien, die Sie in der Colab-Sitzung hochladen oder erzeugen, liegen im temporären Speicher der Laufzeit und verschwinden mit ihr — laden Sie solche Dateien bei Bedarf herunter oder legen Sie sie in Drive ab. Lokal und auf dem Hochschul-Server speichert JupyterLab regelmäßig selbst; **Strg+S** (Mac: **Cmd+S**) erzwingt es.

Die Spielregeln der Kurs-Notebooks. Prü fzellen führen Sie einfach aus — sie melden „Alle Prüfungen bestanden ✓“ oder sagen Ihnen, was nicht stimmt. Vorgabe-Zellen sind gegen versehentliches Löschen gesperrt. Und wo `# TODO: Ihre Lösung hier` steht, sind Sie dran.

1.7 Selbsttest: Was kann meine Umgebung?

Führen Sie die folgenden Zellen in jeder Umgebung aus, die Sie nutzen wollen. Sie ändern nichts an Ihrem System — sie schauen nur nach und berichten.

```
# Prü fzelle 1: Ist die Python-Version neu genug für den Kurs? (mindestens 3.10)
① import sys
② version = sys.version_info
③ print(f"Python {version.major}.{version.minor}.{version.micro}")
④ if (version.major, version.minor) >= (3, 10):
    print("✓ Version passt für den Kurs")
else:
    print("× Bitte Python 3.10 oder neuer verwenden (in Colab automatisch erfüllt)")
```

- ① `import sys` lädt ein Modul, also eine fertige Werkzeugsammlung — `sys` enthält Informationen über die laufende Python-Installation (mehr zu Modulen im Python-Grundkurs)
- ② `sys.version_info` liefert die Versionsnummer; der Punkt greift auf einen benannten Bestandteil zu (ein „Attribut“): `.major` ist die Hauptversion, `.minor` die Nebenversion
- ③ das `f` vor dem String setzt die Werte in `{}` direkt in den Text ein (f-String — Kapitel „Strings“ im Python-Grundkurs)
- ④ Werte in runden Klammern bilden ein Paar (Tupel); Python vergleicht Paare elementweise: erst die Hauptversion, bei Gleichstand die Nebenversion — so ist 3.9 korrekt kleiner als 3.10

```

# Prüfwelle 2: Sind die Kurs-Pakete da? torch darf anfangs fehlen (kommt später im Semester)
import importlib
① for package in ["numpy", "pandas", "matplotlib", "torch"]:
②     try:
③         module = importlib.import_module(package)
④         print(f"✓ {package:12s} {getattr(module, '__version__', '?')}")
⑤     except ModuleNotFoundError:
⑥         hints = {"torch": "kommt später im Semester dazu"}
         hint = hints.get(package, "fehlt – bitte melden")
         print(f"– {package:12s} nicht installiert ({hint})")
⑦     except ImportError as fehler:
         print(f"x {package:12s} Import fehlgeschlagen: {fehler}")

```

- ① `for package in [...]` läuft nacheinander über die Einträge der Liste; `package` ist in jedem Durchlauf der aktuelle Paketname (Schleifen: Python-Grundkurs)
- ② `try` probiert den eingerückten Block aus; geht darin etwas schief, bricht das Programm nicht ab, sondern springt zum passenden `except` weiter unten
- ③ `importlib.import_module(name)` tut dasselbe wie `import`, nimmt den Modulnamen aber als Text entgegen — nur so funktioniert er mit der Variablen `package`
- ④ `getattr(objekt, name, ersatz)` liest ein Attribut (einen benannten Bestandteil) über seinen Namen — hier die Versionsnummer `__version__`; hat ein Paket keine, kommt der Ersatzwert `'?'`
- ⑤ `except ModuleNotFoundError` fängt genau den Fall „Modul nicht installiert“ ab — statt Absturz läuft es hier weiter
- ⑥ `hints` ist ein Dictionary, eine Zuordnung von Schlüsseln zu Werten in geschweiften Klammern; `.get(schlüssel, ersatz)` schlägt nach und liefert den Ersatz, wenn der Schlüssel fehlt (Dictionaries kommen früh im Semester ausführlich dran)
- ⑦ `except` darf mehrfach stehen, Python nimmt den ersten passenden: Jeder andere `ImportError` heißt hier „installiert, aber beschädigt“ — `as fehler` hebt die Originalmeldung zum Ausgeben auf

```

# Prüfwelle 3: Steht eine CUDA-GPU zur Verfügung? (ohne GPU kein Fehler – nur eine Auskunft)
try:
    import torch
① if torch.cuda.is_available():
        print(f"✓ CUDA-GPU gefunden: {torch.cuda.get_device_name(0)}")
        print(" Die GPU-Demos der bisherigen Lerneinheiten laufen bei Ihnen live.")
    else:
        print("– PyTorch da, aber in dieser Python-Umgebung ist CUDA nicht verfügbar")
        print(" (keine NVIDIA-Karte, fehlender Treiber oder eine CPU-Version von PyTorch).")
        print(" Kein Problem: CPU reicht für die ersten Lerneinheiten;")
        print(" für GPU-Teile nutzen Sie Colab (Laufzeit -> T4-GPU) oder den Hochschul-Server.")
② except ModuleNotFoundError:
    print("– Ohne PyTorch keine GPU-Prüfung; in Colab ist PyTorch vorinstalliert.")
except ImportError as fehler:
    print("x PyTorch-Import fehlgeschlagen:", fehler)

```

- ① `torch.cuda.is_available()` fragt PyTorch, ob es eine nutzbare CUDA-GPU sieht — die Punkte hängen sich durchs Paket: Modul `torch`, Teilbereich `cuda`, Funktion `is_available`
- ② `except ModuleNotFoundError` fängt den Fall ab, dass PyTorch nicht installiert ist — statt eines Fehlers erscheint dann nur die Meldung darunter; ein anderer `ImportError` (zweiter Block) heißt „installiert, aber beschädigt“

So lesen Sie das Ergebnis: In **Colab** sollten alle drei Zellen grün sein (die GPU-Zeile nur mit eingeschalteter T4-Laufzeit). **Lokal** über das Startskript fehlt planmäßig nur `torch` — es ist groß und kommt erst später im Semester in die Kursumgebung. Meldet Prüfwelle 2 dagegen `numpy`, `pandas` oder `matplotlib` als fehlend, stimmt mit der Umgebung etwas nicht: Starten Sie über das Startskript (Weg 2) statt über ein eigenes Jupyter, oder fragen Sie im Termin nach.